

Avaliação Empírica da Maturidade de Testes em *Pipelines* CI/CD e seu Impacto na Qualidade de Software

Eric Rodrigues Diniz¹

¹Bacharelado em Engenharia de Software

Instituto de Ciências Exatas e Informática – PUC Minas

Rua Cláudio Manoel, 1162, Funcionários, Belo Horizonte – MG – Brasil

`erdiniz@sga.pucminas.br`

Abstract. *Continuous Integration and Continuous Delivery (CI/CD) pipelines have become central to modern software development, but the adoption of automated testing practices varies substantially among open-source projects. This study empirically evaluates the maturity of testing practices in GitHub Actions pipelines and analyzes their relationship with software quality and maintenance indicators. The research follows a quantitative approach based on repository mining, metric extraction through GitHub's API, and statistical analysis. In total, 518 repositories were initially mapped, of which 477 used GitHub Actions and composed the analyzed sample. The data collection and processing pipeline calculated three maturity indices: Pipeline Maturity Index (PMI), Testing Maturity Index (TMI), and Integrated Maturity Index (IMI). The clustering results classified 254 repositories as robust (53.2%), 221 as fragile (46.3%), and 2 as intermediate (0.4%). Repositories in the robust group presented a higher average test pass rate (80.3%) than those in the fragile group (5.9%), suggesting that higher CI/CD testing maturity is associated with better operational quality indicators. The study contributes by providing an empirical characterization of testing maturity in CI/CD pipelines and by proposing maturity indices to support comparative analyses between repositories.*

Resumo. *Pipelines de Integração Contínua e Entrega Contínua (CI/CD) tornaram-se centrais no desenvolvimento moderno de software, mas a adoção de práticas de teste automatizado ainda varia significativamente entre projetos open-source. Este trabalho avalia empiricamente a maturidade das práticas de teste em pipelines do GitHub Actions e analisa sua relação com indicadores de qualidade e manutenção de software. A pesquisa adota uma abordagem quantitativa, baseada em mineração de repositórios, extração de métricas por meio da API do GitHub e análise estatística dos dados. Ao todo, foram mapeados 518 repositórios, dos quais 477 utilizavam GitHub Actions e compuseram a amostra analisada. O pipeline de coleta e processamento calculou três índices de maturidade: Pipeline Maturity Index (PMI), Testing Maturity Index (TMI) e Integrated Maturity Index (IMI). Os resultados da clusterização classificaram 254 repositórios como robustos (53,2%), 221 como frágeis (46,3%) e 2 como intermediários (0,4%). Os repositórios do grupo robusto apresentaram taxa média de aprovação dos testes de 80,3%, enquanto o grupo frágil apresentou 5,9%, sugerindo que maior maturidade de testes em CI/CD está associada a melhores*

indicadores operacionais de qualidade. O estudo contribui ao caracterizar empiricamente a maturidade de testes em pipelines CI/CD e ao propor índices de maturidade para apoiar análises comparativas entre repositórios.

Bacharelado em Engenharia de Software - PUC Minas
Trabalho de Conclusão de Curso (TCC)

Orientador de conteúdo (TCC I): Danilo Maia – danilomaia@pucminas.br
Orientador de conteúdo (TCC I): Leonardo Vilela – leonardocardoso@pucminas.br
Orientador de conteúdo (TCC I): Raphael Ramos – rrdcostasi@gmail.com
Orientador acadêmico (TCC I): Cleiton Tavares – cleitontavares@pucminas.br
Orientador do TCC II: Danilo Maia – danilomaia@pucminas.br

Belo Horizonte, 14 de junho de 2026.

1. Introdução

A crescente complexidade dos sistemas de *software* tem impulsionado a adoção de práticas automatizadas voltadas ao aumento da qualidade, da eficiência e da confiabilidade das entregas. Nesse contexto, a cultura *DevOps* e os *pipelines* de Integração Contínua e Entrega Contínua (CI/CD) tornaram-se fundamentais para sustentar fluxos modernos de desenvolvimento e manutenção contínua. Esses fluxos permitem que mudanças sejam integradas, verificadas e disponibilizadas com maior frequência, tornando os testes ainda mais relevantes. De acordo com Sommerville (2011), o teste de *software* é essencial para verificar se o sistema atende aos requisitos e revelar comportamentos incorretos ou indesejados. Em ambientes CI/CD, os testes automatizados passam a integrar o próprio fluxo dos *pipelines*, influenciando diretamente a entrega de *software* com qualidade em ciclos curtos.

Embora a automação de testes seja central em ambientes de CI/CD, sua simples adoção não garante automaticamente maior qualidade do *software* ou menor custo de manutenção. Wang et al. (2022) indicam que níveis mais elevados de maturidade em automação de testes estão associados à melhoria da qualidade do produto e à redução do ciclo de entrega, enquanto Wiklund et al. (2012) mostram que práticas pouco padronizadas, *scripts* frágeis e ambientes de teste mal estruturados podem gerar dívida técnica. De forma complementar, Valenzuela-Toledo et al. (2024) evidenciam que *workflows* de automação demandam manutenção recorrente, e Reich e Maalej (2023) observam que ajustes de testabilidade aparecem com frequência em mudanças de código. Assim, permanece o problema de compreender até que ponto a maturidade dos testes em *pipelines* CI/CD contribui para reduzir falhas sem tornar o esforço de manutenção desproporcional. A questão de pesquisa que orienta este trabalho é: **Qual o nível de maturidade de testes em pipelines CI/CD necessário para reduzir falhas em produção sem elevar excessivamente os custos de manutenção?**

Compreender essa relação é fundamental para equilibrar qualidade, custo e sustentabilidade em ambientes de desenvolvimento contínuo. Valenzuela-Toledo et al. (2024) mostraram que arquivos de *workflow* representam apenas 2,1% dos repositórios, mas exigem manutenção constante em 78% dos casos, evidenciando custos ocultos associados à automação. De forma complementar, Feio et al. (2024) demonstraram que a integração de

testes contínuos em abordagens *DevSecOps* reduziu em 37% as vulnerabilidades não detectadas em fases iniciais, reforçando o papel da maturidade dos *pipelines* como elemento relevante para a qualidade e a segurança do *software*. Nesse sentido, este estudo busca contribuir para a redução da lacuna existente na literatura sobre o impacto da maturidade de testes na confiabilidade dos sistemas e no esforço de manutenção da automação.

Diante desse contexto, este trabalho tem como **objetivo geral avaliar a maturidade de testes em *pipelines* CI/CD e seu impacto na qualidade do *software* em projetos open-source que utilizam o *GitHub Actions***. Esse objetivo é detalhado a partir dos seguintes objetivos específicos: (i) comparar níveis de qualidade de *software* em projetos com diferentes graus de maturidade de testes nos *pipelines* CI/CD; (ii) investigar a relação entre a frequência e o tipo de execução de testes automatizados e a ocorrência de falhas em produção; e (iii) analisar o custo de manutenção associado a diferentes graus de maturidade de testes, examinando os *trade-offs* entre esforço e confiabilidade. Como resultado, foram produzidas evidências empíricas sobre a relação entre maturidade de testes em *pipelines* CI/CD, qualidade do *software* e esforço de manutenção em projetos *open-source* que utilizam o *GitHub Actions*. Para isso, foram considerados os repositórios coletados na amostra e os índices sintéticos de maturidade definidos no estudo, de modo a avaliar diferentes dimensões da estrutura dos *pipelines* e das práticas de teste automatizado. Com isso, o trabalho buscou apoiar a compreensão dos *trade-offs* entre automação, confiabilidade e custo operacional em ambientes de desenvolvimento contínuo.

O trabalho encontra-se organizado da seguinte forma: a Seção 2 apresenta os fundamentos teóricos relacionados à maturidade de testes e à automação em *pipelines* CI/CD; a Seção 3 discute os trabalhos relacionados e posiciona esta pesquisa no contexto atual da Engenharia de Software; a Seção 4 descreve a metodologia adotada, incluindo critérios de seleção, coleta e análise dos dados; a Seção 5 apresenta a visualização geral dos resultados; a Seção 6 discute os resultados à luz dos objetivos e questões de pesquisa; a Seção 7 apresenta as ameaças à validade do estudo; e, por fim, a Seção 8 traz as conclusões e recomendações para trabalhos futuros.

2. Fundamentação Teórica

Esta seção apresenta os conceitos que sustentam o estudo: testes automatizados em CI/CD, qualidade e maturidade em *pipelines* e aspectos de manutenção e dívida técnica associados à automação.

2.1. Testes em CI/CD

Os testes de *software* são fundamentais para verificar se um sistema atende aos requisitos definidos e identificar comportamentos incorretos ou indesejados ao longo do desenvolvimento [Sommerville 2011, Pressman and Maxim 2016]. Em ambientes de Integração Contínua e Entrega Contínua (CI/CD), eles deixam de ser executados apenas em etapas isoladas e passam a integrar o fluxo automatizado de validação do projeto. A Integração Contínua pressupõe a incorporação frequente de mudanças ao repositório principal, acompanhada de verificações automatizadas, como compilação, execução de testes e outras validações [Fowler 2006]. A Entrega Contínua amplia essa lógica ao manter o *software* em estado continuamente implantável, o que depende de *pipelines* capazes de operar com verificações confiáveis ao longo do ciclo de entrega [Humble and Farley 2010]. Nesse

contexto, os testes automatizados sustentam entregas rápidas, frequentes e confiáveis. Entretanto, sua efetividade depende da forma como são estruturados e mantidos, pois testes instáveis, frágeis ou pouco representativos podem reduzir a confiança no *pipeline* e ampliar o esforço de manutenção.

2.2. Qualidade e Maturidade em *Pipelines CI/CD*

A qualidade de *software* pode ser compreendida como a capacidade de um sistema atender aos requisitos funcionais e não funcionais esperados, incluindo atributos como confiabilidade, manutenibilidade e eficiência [Sommerville 2011, iso 2011]. Neste trabalho, ela é observada por meio de indicadores relacionados à execução dos *pipelines*, à aprovação dos testes, à ocorrência de falhas e à estabilidade dos mecanismos automatizados de validação. A maturidade de um *pipeline* corresponde ao grau em que o fluxo de *CI/CD* está estruturado, automatizado e preparado para executar validações de forma estável e recorrente, envolvendo aspectos como organização dos *workflows*, quantidade e complexidade dos *jobs*, uso de matriz de execução, presença de verificações automatizadas, ferramentas de cobertura, análise estática, mecanismos de segurança e manutenção dos arquivos de configuração [Valenzuela-Toledo et al. 2024]. A maturidade de testes, por sua vez, representa o grau em que as práticas de teste estão presentes, organizadas e integradas ao processo de desenvolvimento, podendo ser observada por elementos como volume de arquivos de teste, presença de testes unitários, de integração e de ponta a ponta, diversidade de *frameworks*, proporção de testes no repositório, taxa de aprovação e ocorrência de testes instáveis [Wang et al. 2022]. Neste trabalho, a maturidade de testes em *pipelines CI/CD* é entendida como a integração entre a estrutura do *pipeline* e a qualidade das práticas de teste automatizado, permitindo avaliar se repositórios com *pipelines* mais estruturados e testes mais consistentes apresentam melhores indicadores de qualidade, estabilidade e manutenção [Wang et al. 2022].

2.3. Manutenção e Dívida Técnica de Testes em *Pipelines CI/CD*

Embora a automação seja central em ambientes de *CI/CD*, ela também introduz artefatos que precisam ser mantidos continuamente, como arquivos de *workflow*, configurações de ambiente, dependências, *scripts* de teste e mecanismos automatizados de validação. Valenzuela-Toledo et al. (2024) mostram que a manutenção de *workflows* em GitHub Actions pode gerar custos ocultos, pois esses artefatos evoluem em conjunto com o código de produção e com os testes automatizados [Valenzuela-Toledo et al. 2024]. A literatura também evidencia que problemas nos artefatos de teste comprometem a estabilidade dos *pipelines*. Testes intermitentes, ou *flaky tests*, apresentam resultados inconsistentes mesmo sem alterações no código sob teste, reduzindo a confiança das equipes na automação e elevando o esforço de depuração [Luo et al. 2014]. De forma complementar, a dívida técnica de testes, ou *test debt*, está associada à degradação estrutural dos testes ao longo do tempo, incluindo baixa padronização, presença de *test smells*, excesso de testes frágeis e dificuldade de manutenção [Bavota et al. 2015, Rizzi et al. 2020]. Em *pipelines CI/CD*, esses problemas afetam diretamente a previsibilidade das execuções e a capacidade de entregar *software* com qualidade. Práticas como refatoração para testabilidade, melhoria contínua dos testes e organização dos artefatos automatizados são apontadas como mecanismos relevantes para reduzir instabilidade e aumentar a confiabilidade dos fluxos de entrega [Reich and Maalej 2023]. Dessa forma, a manutenção dos

testes e dos *workflows* é tratada neste trabalho como fator operacional relevante para compreender a maturidade dos *pipelines* e sua relação com a qualidade observada nos projetos analisados.

3. Trabalhos Relacionados

Valenzuela-Toledo et al. (2024) investigaram a manutenção de *workflows* do GitHub Actions em 183 projetos maduros do GitHub, sendo 27 com apenas um *workflow* e 156 com múltiplos *workflows*. Os autores observaram que esses arquivos representam pequena parcela dos repositórios, com frequência média de 3,05% e 3,74%, respectivamente, mas apresentam taxa média mensal de alteração de 6,9% e 8,2%. A análise de acoplamento mostrou que, quando um *workflow* é modificado, há 12,26% de chance de alteração conjunta em arquivos de produção no grupo com *workflow* único e 17,00% no grupo com múltiplos *workflows*. Na análise qualitativa, melhorias de *CI/CD* explicaram 61% dos casos de acoplamento entre *workflow* e teste e 33% dos casos entre *workflow* e produção. Esses resultados evidenciam que a automação em *CI/CD* introduz artefatos sujeitos à evolução e manutenção contínuas.

Wang et al. (2022) conduziram um estudo quantitativo com 37 projetos Java *open-source* que utilizavam integração contínua. Os autores avaliaram qualidade do produto, esforço de automação de testes e ciclo de *release*. A maturidade de automação apresentou associação significativa com menor densidade de defeitos, com coeficiente de -5,070e-02 e valor-p de 0,000624, e também com aceleração do ciclo de *release*, com coeficiente de 4,135e-02 e valor-p de 0,01891. Em contrapartida, não houve evidência estatística de aumento proporcional do esforço de automação, já que a variável maturidade apresentou valor-p de 0,8611 no modelo relacionado ao esforço. Esses achados indicam que práticas maduras de automação podem estar associadas à melhoria da qualidade e à redução do tempo de entrega sem, necessariamente, elevar o esforço de automação.

Reich e Maalej (2023) investigaram padrões de refatoração voltados à testabilidade em projetos Java hospedados no GitHub. O estudo partiu de 508.240 *pull requests*, dos quais 72.115 continham pares entre código de produção e código de teste. Na análise automatizada, foram observadas 273.201 refatorações em 7.822 *pull requests* com pares de teste, contra 114.427 em 4.839 *pull requests* sem pares de teste. Na análise manual de 724 *pull requests*, cerca de 25% continham padrões de refatoração voltados à testabilidade, totalizando 230 ocorrências. Esses resultados mostram que a melhoria da testabilidade exige mudanças recorrentes no código de produção e nos testes.

Outros estudos contribuem principalmente no plano metodológico. Nwosu et al. (2024) propõem um modelo de mineração de repositórios com *k-Nearest Neighbor* para classificar informações extraídas de código-fonte, destacando ganhos de velocidade e organização dos dados. Abdallah e Alrifae (2022) desenvolvem a ferramenta *Java Code Quality Reviewer* para mensurar qualidade de código Java por meio de regras heurísticas de inspeção estática. Alaswad e Poovammal (2022), por sua vez, revisam abordagens de previsão de qualidade com técnicas de *machine learning* e métricas de produto, processo e projeto. Embora não tratem diretamente de *CI/CD*, esses trabalhos reforçam a viabilidade de analisar qualidade e manutenção a partir de métricas objetivas extraídas de dados reais.

Em conjunto, esses estudos mostram que qualidade, maturidade e manutenção

podem ser analisadas por meio de métricas empíricas. Diferentemente dos trabalhos anteriores, esta pesquisa integra maturidade de *pipelines*, práticas de teste automatizado, indicadores de qualidade e esforço de manutenção em repositórios *open-source* que utilizam *GitHub Actions*.

4. Materiais e Métodos

Esta seção descreve os materiais, o ambiente de pesquisa e os métodos adotados para avaliar empiricamente a maturidade de testes em *pipelines CI/CD* e sua relação com a qualidade do *software*. A pesquisa foi empírica, de abordagem quantitativa e com objetivos exploratórios, descritivos e explicativos. Quanto ao procedimento técnico, caracterizou-se como estudo empírico observacional baseado em mineração de repositórios de *software*; quanto ao tempo, combinou análise transversal com análise longitudinal complementar. A metodologia baseou-se na coleta automatizada de dados em repositórios *open-source* do GitHub e na análise de métricas de maturidade, qualidade e manutenção.

4.1. Ambiente e Ferramentas de Apoio

Os experimentos foram conduzidos em ambiente local com *scripts* em *Python 3.11*, executados em *macOS M1* com 8 GB de RAM, utilizando o *Visual Studio Code* como ambiente de desenvolvimento. A coleta foi realizada por meio da *API GraphQL do GitHub*, com armazenamento dos dados em arquivos *JSON* e, quando necessário, em *MongoDB Atlas*. O processamento estatístico utilizou as bibliotecas *Pandas*, *NumPy*, *SciPy* e *Scikit-Learn*, enquanto as visualizações foram geradas com *Matplotlib*, *Seaborn* e *Plotly*.

4.2. Procedimentos Metodológicos

A pesquisa seguiu um fluxo metodológico estruturado em seis etapas, apresentado na Figura 1. O processo contemplou a seleção dos repositórios, a coleta dos dados, a classificação da maturidade, a normalização das métricas, a análise estatística e a revisão manual dos resultados.

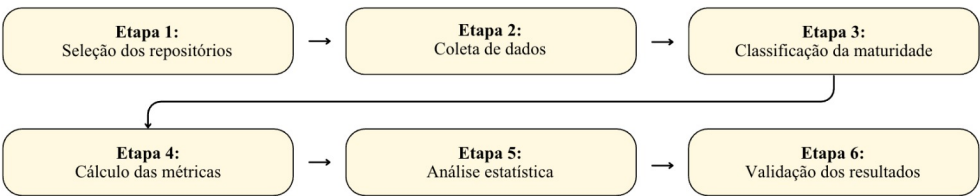


Figura 1. Fluxo das etapas metodológicas do estudo

Na etapa 1, foi realizada a seleção automatizada de repositórios *open-source* no GitHub por meio da *API REST*. Foram considerados repositórios públicos com, no mínimo, 40 mil estrelas, que não fossem *fork*, não estivessem arquivados e apresentassem atividade recente desde janeiro de 2024. A aplicação desses critérios resultou em 518 repositórios inicialmente mapeados, dos quais 477 utilizavam efetivamente o *GitHub Actions* e compuseram a amostra analisada. Esses filtros buscaram reduzir a inclusão de repositórios experimentais, descontinuados ou pouco representativos, mantendo o foco em projetos ativos e com uso observável de automação em *CI/CD*.

Na etapa 2, foi realizada a coleta automatizada dos dados via *API GraphQL do GitHub*, permitindo extrair metadados de *commits*, *issues*, *pull requests*, arquivos de *workflow* e diretórios de teste. Os dados brutos foram armazenados em formato *JSON*, preservando rastreabilidade e reprodutibilidade. Essa etapa permitiu observar o estado dos repositórios no momento da coleta e recuperar sinais de evolução recente, como atividade, *churn* e execuções observáveis.

Na etapa 3, procedeu-se à classificação da maturidade dos *workflows* por meio de heurísticas voltadas à identificação de características como testes automatizados, cobertura, *linting*, análise estática, *build matrix* e mecanismos de segurança. Com base nesses elementos, foram calculados os índices sintéticos de maturidade do estudo, voltados à representação da maturidade estrutural dos *pipelines* e das práticas de teste automatizado. A adoção de heurísticas estruturadas justificou-se pelo caráter composto da maturidade de automação.

Na etapa 4, as métricas coletadas foram transformadas e normalizadas para compor um índice integrado de maturidade. Para isso, foram derivadas métricas relativas e, quando necessário, foi aplicada transformação logarítmica para reduzir a influência de valores extremos. Em seguida, os valores foram normalizados em escala de 0 a 10, permitindo a comparação entre repositórios distintos. Esse procedimento foi necessário porque a amostra apresentou heterogeneidade em porte, atividade e volume de artefatos.

Na etapa 5, foram aplicadas técnicas estatísticas, como correlações de Pearson e Spearman e regressão linear simples, com o objetivo de investigar a relação entre o índice integrado de maturidade e indicadores de qualidade do *software*, como falhas, *bugs*, tempo médio de resolução e atividade recente. Além da análise transversal, foi realizada uma análise temporal em subamostra elegível, considerando janelas temporais equivalentes antes e depois de um marco t_0 , como a introdução de *workflows* ou de artefatos de teste. Essa comparação buscou observar possíveis variações nos indicadores de qualidade e manutenção após a adoção ou intensificação da automação.

Por fim, na etapa 6, os resultados passaram por revisão manual, com conferência dos dados extraídos em relação aos arquivos reais de *workflow*. Essa verificação reduziu falsos positivos e reforçou a consistência dos indicadores adotados, especialmente em um estudo que combinou classificação heurística e mineração automatizada.

4.3. Modelo GQM (Goal-Question-Metric)

Este estudo adota o modelo *Goal-Question-Metric* (GQM) para relacionar objetivos específicos, questões de pesquisa e métricas analisadas. A Tabela 1 apresenta essa estrutura, indicando as perguntas derivadas de cada objetivo e as métricas utilizadas na análise.

Objetivo Específico	Questão	Métricas
Comparar níveis de maturidade de <i>pipelines</i> e testes automatizados com indicadores de qualidade dos projetos analisados.	Q1a: Repositórios com maior maturidade de <i>pipeline</i> apresentam menor densidade de falhas reportadas?	<i>PMI, bugsPerCommit, bugsOpen, bugsClosed.</i>
	Q1b: Repositórios com maior maturidade de testes apresentam maior taxa de aprovação e menor instabilidade nas execuções?	<i>TMI, testPassRate, flakyRate.</i>
	Q1c: Repositórios com maior maturidade integrada apresentam menor tempo médio de resolução de falhas?	<i>IMI, mtrBugHours, bugsClosed.</i>
Caracterizar a estrutura dos <i>pipelines</i> e a composição das práticas de teste automatizado nos repositórios analisados.	Q2a: Repositórios com maior quantidade de <i>workflows</i> e <i>jobs</i> apresentam maior complexidade estrutural de <i>pipeline</i> ?	<i>workflowCount, jobsPerWorkflow, pipelineComplexity.</i>
	Q2b: Repositórios com maior presença de testes unitários, de integração e de ponta a ponta apresentam maior maturidade de testes?	<i>unitTestFiles, integrationTestFiles, e2eTestFiles, testFileCount, testRatio, TMI.</i>
	Q2c: Repositórios com maior diversidade de ferramentas de teste e validação tendem a formar perfis mais robustos de maturidade?	<i>testFrameworkCount, hasCoverageTool, hasLint, hasSAST, PMI, TMI, IMI.</i>
Analisar os <i>trade-offs</i> entre maturidade, estabilidade de testes e esforço de manutenção dos <i>pipelines</i> .	Q3a: Repositórios com maior maturidade integrada apresentam menor ocorrência de testes instáveis?	<i>IMI, flakyRate, testPassRate.</i>
	Q3b: Repositórios com maior frequência de alterações nos <i>workflows</i> apresentam maior esforço de manutenção?	<i>wfChurnLast90d, mtrBugHours, bugsPerCommit.</i>
	Q3c: Repositórios com maior presença de testes por <i>pull request</i> apresentam menor densidade de falhas?	<i>testsPerPR, bugsPerCommit, testPassRate.</i>

Tabela 1. Modelo GQM derivado dos objetivos específicos da pesquisa

4.4. Métricas e Indicadores de Avaliação

As métricas adotadas foram organizadas em dimensões analíticas de caracterização, maturidade, qualidade e manutenção.

Dimensão / Categoria	Métricas	Descrição
Caracterização da Amostra	<i>language</i>	Caracteriza a distribuição dos repositórios por linguagem principal.
Maturidade de Pipelines (PMI)	<i>workflowCount, jobsPerWorkflow, pipelineComplexity, hasMatrix, hasLint, hasCoverageTool, hasSAST, wfChurnLast90d</i>	Avaliam estrutura, complexidade, automação, segurança e evolução recente dos <i>pipelines</i> .
Maturidade de Testes (TMI)	<i>unitTestFiles, integrationTestFiles, e2eTestFiles, testFileCount, testRatio, testFrameworkCount, testPassRate, flakyRate, testsPerPR</i>	Medem volume, diversidade, densidade e estabilidade das práticas de teste automatizado.
Qualidade do Software e Manutenção	<i>bugsOpen, bugsClosed, bugsPerCommit, mtrBugHours</i>	Relacionam maturidade à ocorrência de falhas, densidade de defeitos e esforço corretivo.
Índices Sintéticos	<i>PMI, TMI, IMI</i>	Representam os níveis de maturidade em escala comum.

Tabela 2. Categorias de métricas avaliadas no estudo

A partir das métricas da Tabela 2, foram definidos três índices de maturidade: o

Pipeline Maturity Index (PMI), o *Testing Maturity Index* (TMI) e o *Integrated Maturity Index* (IMI). O PMI sintetiza a maturidade estrutural e operacional dos *pipelines*; o TMI sintetiza a maturidade das práticas de teste automatizado; e o IMI representa a maturidade integrada do repositório, combinando as dimensões de *pipeline* e testes. A formulação completa encontra-se na subseção “Formulação dos Índices de Maturidade” do Apêndice.

5. Resultados

Esta seção apresenta os resultados obtidos a partir da coleta, processamento e análise das métricas dos repositórios selecionados. Inicialmente, descreve-se a sumarização da amostra e das principais métricas analisadas. Em seguida, são apresentadas as visualizações exploratórias dos resultados. Por fim, os achados são discutidos à luz dos objetivos específicos e das questões definidas no modelo GQM.

5.1. Sumarização de Resultados

A amostra foi obtida por meio de coleta automatizada nas APIs REST e GraphQL do GitHub, considerando repositórios públicos com pelo menos 40.000 estrelas, não classificados como *fork*, não arquivados e com atividade recente entre janeiro de 2024 e março de 2026. Após a aplicação dos critérios de seleção e da verificação de uso do *GitHub Actions*, a amostra final utilizada para o cálculo das métricas e dos índices de maturidade foi composta por 477 repositórios. A Tabela 3 resume o processo de filtragem adotado.

ID	Etapas de Seleção	Quantidade
C1	Repositórios retornados pela busca inicial com critérios combinados	518
C2	Repositórios com uso detectado de <i>GitHub Actions</i>	477
C3	Repositórios com métricas coletadas	477
C4	Repositórios com índices PMI, TMI e IMI calculados	477

Tabela 3. Processo de coleta e seleção da amostra de repositórios

A Tabela 4 sintetiza as principais métricas analisadas. Observa-se que PMI, TMI e IMI se concentram em faixas intermediárias, enquanto *bugsPerK*, *mtrBugHours* e *testFileCount* apresentam elevada dispersão, refletindo a heterogeneidade da amostra e justificando o uso de transformações logarítmicas nas visualizações.

Métrica	Média	Mediana	Desvio Padrão	Mín	Máx
<i>PMI</i>	3.011	2.324	2.630	0.000	9.452
<i>TMI</i>	2.970	2.709	1.762	1.200	8.032
<i>IMI</i>	2.991	2.734	2.085	0.600	8.072
<i>bugsPerK</i>	9.657	0.640	40.113	0.000	717.609
<i>mtrBugHours</i>	1783.131	4.790	6704.677	0.000	84080.380
<i>testPassRate</i>	0.457	0.400	0.450	0.000	1.000
<i>flakyRate</i>	0.112	0.000	0.170	0.000	1.000
<i>workflowCount</i>	6.994	3.000	8.833	0.000	30.000
<i>testFileCount</i>	775.373	6.000	3354.207	0.000	49731.000

Tabela 4. Estatísticas descritivas das principais métricas analisadas

Os resultados indicam variabilidade suficiente para investigar relações entre maturidade de *pipelines*, maturidade de testes e qualidade do *software*.

5.2. Visualização dos Resultados

A Figura 2 mostra a distribuição dos repositórios por perfil de maturidade e as distribuições dos índices PMI, TMI e IMI. Observa-se predominância dos perfis frágil e robusto, enquanto o grupo intermediário permanece residual, sugerindo polarização entre repositórios com baixa institucionalização da automação e repositórios com estruturas mais consolidadas de *pipeline* e testes.

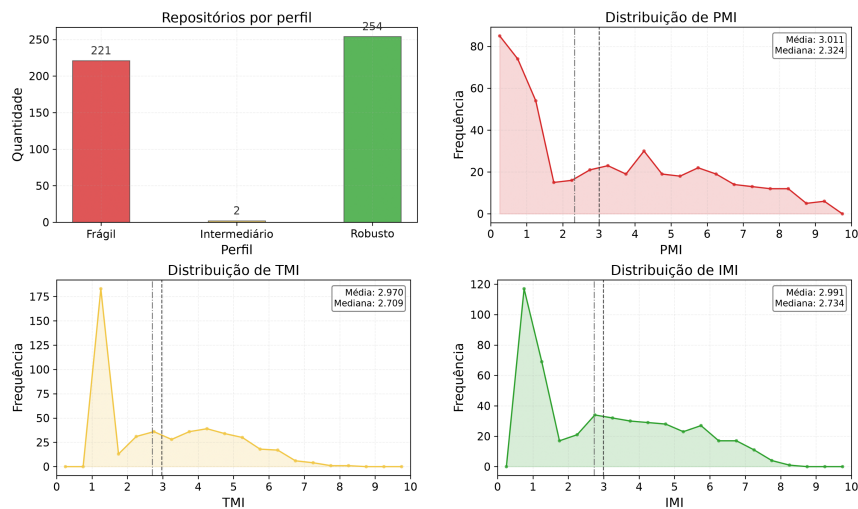


Figura 2. Distribuição geral da amostra e dos índices de maturidade

A Figura 3 reforça essa separação ao apresentar a dispersão dos repositórios no plano PMI versus TMI. Os repositórios frágeis concentram-se em faixas inferiores de ambos os índices, enquanto os robustos ocupam regiões mais elevadas, sustentando a coerência da classificação adotada.

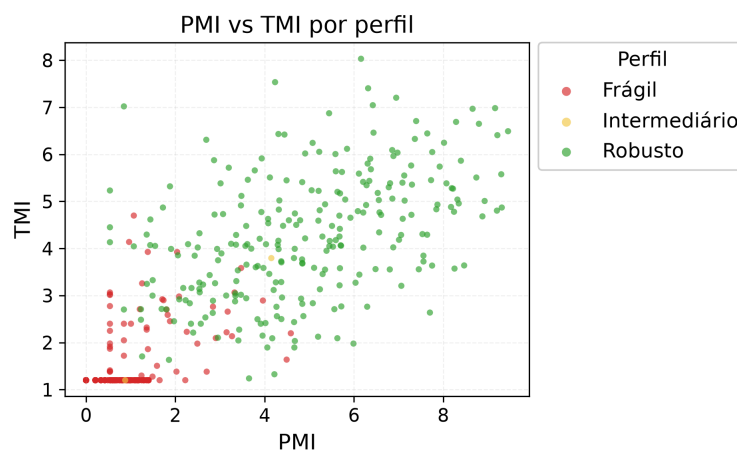


Figura 3. Perfis de maturidade no espaço PMI versus TMI

A Figura 4 reúne as relações entre maturidade e qualidade. Na relação entre IMI e *bugsPerK*, a associação é fraca, indicando que a maturidade não explica isoladamente a densidade de defeitos. Contudo, na análise do *mtrBugHours* por faixa de PMI, os

repositórios robustos tendem a se concentrar em níveis inferiores da métrica transformada. Já na análise de *bugsClosed* por faixa de TMI, permanece visível maior capacidade de encerramento de defeitos em repositórios com maior maturidade de testes.

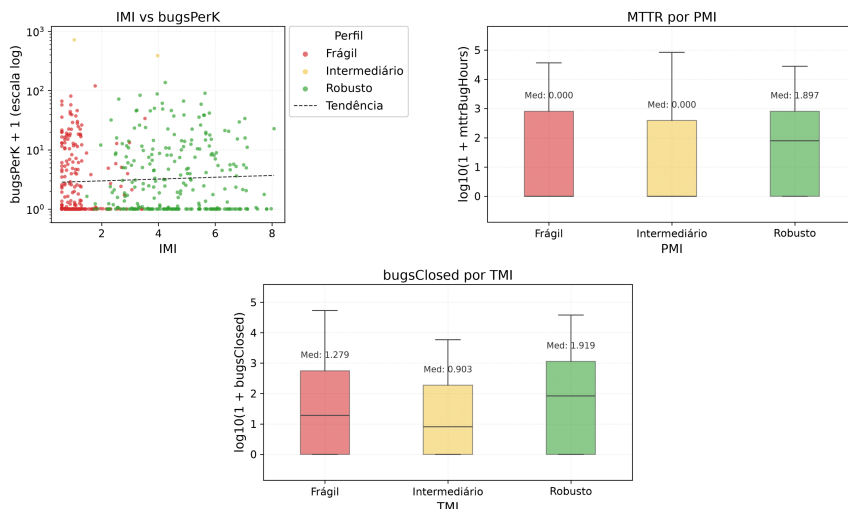


Figura 4. Relações entre maturidade e indicadores de qualidade do *software*

A Figura 5 mostra associação positiva entre *workflowCount* e *pipelineComplexity*, com repositórios robustos concentrando maior número de *workflows* e maior complexidade estrutural. A relação entre *testFileCount* e *testRatio* também diferencia os perfis: os frágeis permanecem em baixos volumes e proporções de testes, enquanto os robustos concentram os maiores valores observados.

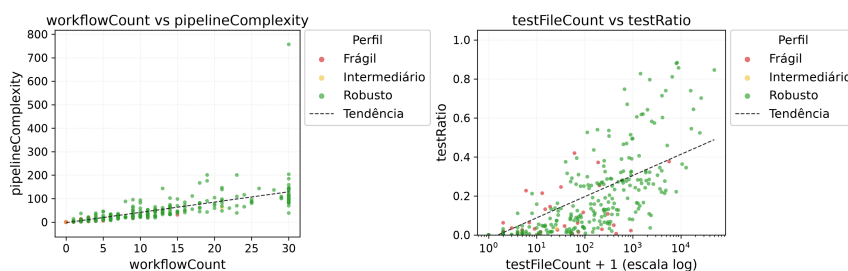


Figura 5. Estrutura dos *pipelines* e densidade da base de testes

A Figura 6 detalha a composição dos testes por perfil, considerando valores transformados em escala logarítmica. O contraste mais evidente ocorre nos testes unitários, cuja distribuição é mais elevada no grupo robusto. Testes de integração e E2E permanecem em faixas inferiores, embora com maior dispersão entre os robustos, sugerindo incorporação gradual de estratégias complementares de verificação.

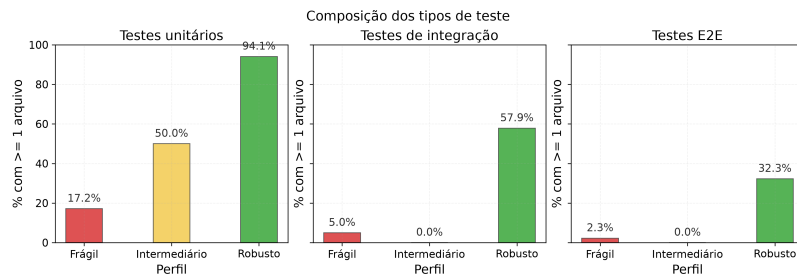


Figura 6. Composição dos testes automatizados por perfil de maturidade

A Figura 7 apresenta os *trade-offs* entre estabilidade, esforço de manutenção e maturidade. Maiores taxas de aprovação coexistem, em parte da amostra, com níveis relevantes de *flakyRate*, indicando que a maturidade não elimina automaticamente testes instáveis. Além disso, a análise de *wfChurnLast90d* por faixa de IMI mostra que o grupo robusto permanece em patamar superior, sugerindo maior esforço recente de manutenção dos *workflows*.

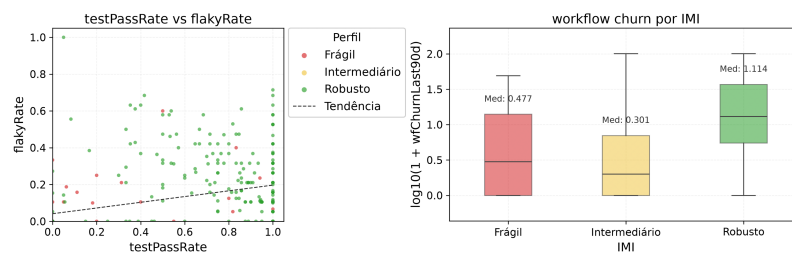


Figura 7. Trade-offs entre estabilidade, manutenção e maturidade

A análise temporal complementar comparou duas janelas de 180 dias em torno do marco temporal definido para cada repositório. A Figura 8 indica aumento de *bugs-Closed* na janela posterior, continuidade do esforço de manutenção dos *workflows*, maior visibilidade da *flakyRate* e ausência de redução uniforme do *mttrBugHours*.

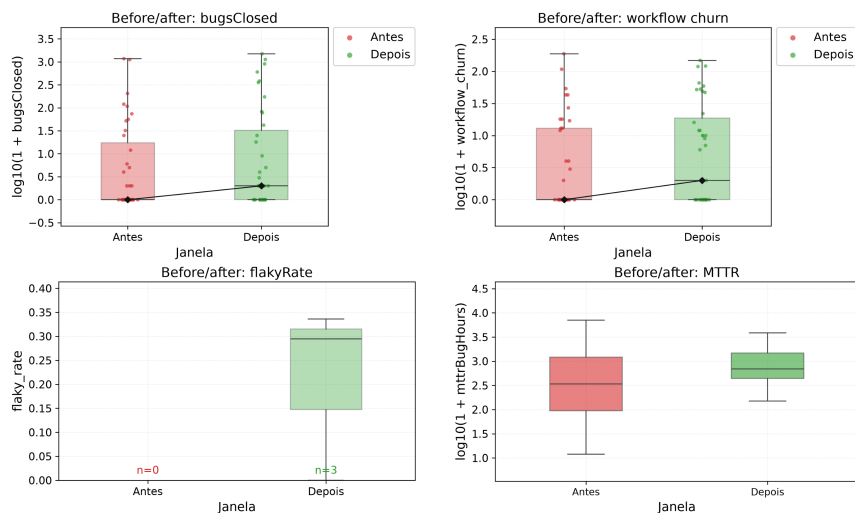


Figura 8. Comparação dos indicadores nas janelas anterior e posterior ao marco temporal

A Figura 9 mostra as distribuições dos deltas de *bugsClosed* e de *MTTR*. Enquanto o delta de *bugsClosed* se concentra majoritariamente em valores positivos, o delta de *MTTR* apresenta maior dispersão, incluindo valores positivos e negativos.

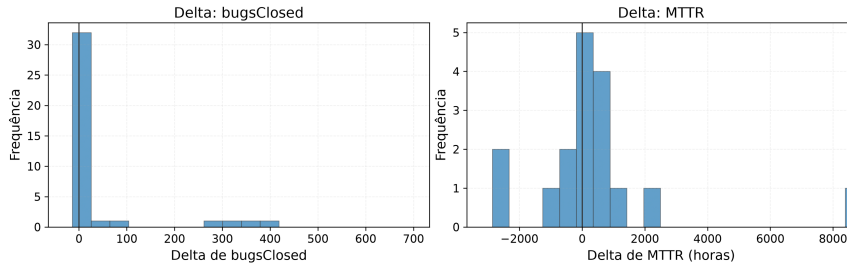


Figura 9. Distribuição dos deltas temporais de qualidade e manutenção

5.3. Discussão dos Resultados

Esta subseção discute os resultados apresentados à luz dos objetivos específicos e das questões definidas no modelo GQM.

5.3.1. Objetivo 1: Relação entre maturidade e qualidade do *software*

Os resultados indicam que a maturidade integrada não apresenta associação linear clara com a densidade de defeitos. A dispersão de *bugsPerK*, observada na Tabela 4 e na relação com o IMI, mostra que repositórios com diferentes níveis de maturidade podem coexistir com valores distintos de defeitos reportados. Assim, os achados não confirmam que maior maturidade, isoladamente, reduza a densidade de falhas (Q1a). Por outro lado, os repositórios robustos tendem a apresentar menores valores relativos de *mttrBugHours* e maiores valores de *bugsClosed*, sugerindo maior capacidade de resposta e encerramento de defeitos (Q1b; Q1c). Portanto, o Objetivo 1 foi atendido parcialmente: há evidências de associação entre maturidade e resposta operacional, mas não de melhoria homogênea em todos os indicadores de qualidade.

5.3.2. Objetivo 2: Estrutura dos *pipelines* e composição dos testes automatizados

O Objetivo 2 apresentou os resultados mais consistentes. Os repositórios robustos concentram maiores valores de *workflowCount* e *pipelineComplexity*, indicando que a maturidade estrutural envolve maior organização, diversificação e complexidade dos fluxos de *CI/CD* (Q2a). De modo semelhante, apresentam maior volume relativo de arquivos de teste e maiores valores de *testRatio*, enquanto os frágeis permanecem em níveis reduzidos dessas métricas (Q2b). A composição por perfil mostra que a principal diferença está na consolidação dos testes unitários, embora os robustos também apresentem maior dispersão em testes de integração e E2E. Além disso, a separação entre os perfis no espaço PMI versus TMI sugere que maior diversidade de práticas e ferramentas de validação contribui para perfis mais consolidados de maturidade (Q2c). Dessa forma, o Objetivo 2 foi amplamente atendido.

5.3.3. Objetivo 3: *Trade-offs* entre maturidade, estabilidade e manutenção

Os resultados do Objetivo 3 indicam que maior maturidade não elimina automaticamente a instabilidade dos testes. Embora repositórios robustos apresentem maiores taxas de aprovação, a dispersão de *flakyRate* mostra que testes instáveis ainda podem ocorrer em ambientes mais maduros (Q3a). Além disso, a análise de *wfChurnLast90d* evidencia que os robustos tendem a apresentar maior esforço recente de manutenção dos *workflows*, sugerindo que maturidade e custo de sustentação podem crescer simultaneamente (Q3b). A análise temporal reforça esse ponto ao mostrar continuidade do *churn* após o marco temporal, sem melhora uniforme nos indicadores operacionais. Assim, a maturidade em *CI/CD* deve ser interpretada como equilíbrio entre cobertura, capacidade operacional, confiabilidade e custo contínuo de sustentação (Q3c).

6. Ameaças à Validade

A Tabela 5 resume as principais ameaças à validade deste estudo e as ações adotadas para reduzir seus efeitos sobre a interpretação dos resultados.

Tipo da Validade	Descrição da Ameaça	Ação
Construto	Os índices PMI, TMI e IMI representam aproximações quantitativas da maturidade de testes em <i>pipelines CI/CD</i> e podem não capturar integralmente aspectos qualitativos do processo, como decisões arquiteturais, práticas informais de revisão e particularidades do contexto de desenvolvimento.	As métricas foram definidas a partir do modelo GQM, organizadas em dimensões separadas e tiveram sua formulação explicitada no Apêndice, buscando maior transparência e rastreabilidade dos indicadores.
Interna	Os resultados podem ter sido influenciados por fatores não controlados, como porte do projeto, domínio da aplicação, popularidade do repositório, intensidade de atividade recente e eventos externos ao marco temporal adotado.	Os achados foram interpretados de forma associativa, sem inferência causal, e a análise utilizou janelas temporais equivalentes, métricas padronizadas e comparação entre perfis de maturidade.
Conclusão	A elevada dispersão de algumas métricas e a dependência de subamostras elegíveis na análise temporal podem limitar a estabilidade estatística e a força das conclusões obtidas.	Foram empregadas estatísticas descritivas, transformações logarítmicas e análise comparativa entre perfis, reduzindo o efeito de assimetrias e favorecendo interpretações mais consistentes dos resultados.
Externa	A amostra é composta por repositórios <i>open-source</i> do GitHub, com alta popularidade, atividade recente e uso detectado do GitHub Actions, o que restringe a generalização para projetos privados, menores ou baseados em outras plataformas de <i>CI/CD</i> .	Os critérios de seleção foram explicitados e o escopo de generalização foi limitado ao contexto analisado; como extensão futura, o estudo pode ser ampliado para outros ambientes e ecossistemas.

Tabela 5. Resumo das ameaças à validade do estudo

7. Conclusão

Este trabalho avaliou empiricamente a maturidade de testes em *pipelines CI/CD* e sua relação com indicadores de qualidade e manutenção em repositórios *open-source* hospedados no GitHub. A análise considerou 518 repositórios inicialmente mapeados, dos quais 477 utilizavam *GitHub Actions* e compuseram a amostra analisada. A partir dessa base, foram construídos os índices PMI, TMI e IMI, voltados, respectivamente, à mensuração da maturidade estrutural dos *pipelines*, da maturidade das práticas de teste automatizado e da maturidade integrada dos repositórios. A clusterização resultou em 254 repositórios robustos, 221 frágeis e 2 intermediários, evidenciando uma polarização entre projetos

com baixa institucionalização da automação e projetos com estruturas mais consolidadas de validação contínua.

Os resultados indicaram que a maturidade de testes em *pipelines CI/CD* não se relaciona de forma linear com todos os indicadores de qualidade analisados. Em especial, a densidade de defeitos não apresentou redução uniforme em função do aumento da maturidade, sugerindo que a ocorrência de falhas também depende de fatores contextuais, como porte, domínio, atividade recente e forma de registro dos problemas nos repositórios. No entanto, os repositórios classificados como robustos apresentaram maior capacidade estrutural e operacional, observada em *pipelines* mais complexos, maior volume e diversidade de testes, menores valores relativos de MTTR em faixas mais maduras e maior volume de *bugs* fechados. Além disso, o grupo robusto apresentou taxa média de aprovação dos testes de 80,3%, enquanto o grupo frágil apresentou 5,9%, reforçando a associação entre maturidade e melhor desempenho operacional da automação.

Por outro lado, os achados também evidenciaram que maior maturidade não elimina automaticamente problemas de instabilidade e pode estar associada a maior esforço de manutenção dos *workflows*. Assim, a maturidade em *CI/CD* deve ser compreendida não apenas como aumento de automação, mas como um equilíbrio entre capacidade de verificação, confiabilidade, estabilidade e custo operacional. Dessa forma, conclui-se que a maturidade de testes em *pipelines CI/CD* contribui para qualificar a infraestrutura de validação e fortalecer a capacidade de resposta dos projetos, embora seus efeitos dependam de fatores contextuais e não possam ser interpretados como causais. Como contribuição, este trabalho oferece uma caracterização empírica da maturidade de testes em *pipelines CI/CD* e propõe os índices PMI, TMI e IMI como instrumentos de apoio à comparação entre repositórios. Como trabalhos futuros, sugere-se ampliar a amostra para outros ecossistemas de *CI/CD*, incluir projetos privados ou corporativos, refinar as métricas de instabilidade e manutenção e aprofundar análises estatísticas capazes de investigar relações causais entre maturidade, qualidade e esforço de sustentação da automação.

Pacote de Replicação

O pacote de replicação deste trabalho encontra-se disponível em:

<https://github.com/ICEI-PUC-Minas-PPLES-TI/plf-es-2025-2-tcci-0393100-pes-eric-diniz>

Referências

- (2011). Iso/iec 25010:2011 - systems and software engineering — systems and software quality requirements and evaluation (square) — system and software quality models. Technical report, ISO, Geneva.
- Abdallah, M. and Alrifaa, M. (2022). A heuristic tool for measuring software quality using program language standards. *International Arab Journal of Information Technology*, 19(3):314–322.
- Alaswad, F. and Poovammal, E. (2022). Software quality prediction using machine learning. *Materials Today: Proceedings*, 62:4714–4720.

- Bavota, G., Linares-Vásquez, M., Bernal-Cárdenas, C., and Poshyvanyk, D. (2015). The impact of test smells on test coverage. In *2015 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 373–383. IEEE.
- Feio, C., Santos, N., Escravana, N., and Pacheco, B. (2024). An empirical study of devsecops focused on continuous security testing. In *2024 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*. IEEE.
- Fowler, M. (2006). Continuous integration. <https://martinfowler.com/articles/continuousIntegration.html>. Acesso em: 17 nov. 2025.
- Humble, J. and Farley, D. (2010). *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley, Boston.
- Luo, Q., Hariri, F., Lam, W., Shi, A., and Marinov, D. (2014). An empirical analysis of flaky tests in automated test suites. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, pages 643–653. ACM.
- Nwosu, P. C., Onuodu, F. E., and Okengwu, U. A. (2024). Enhanced model for mining software repositories. *International Journal of Computer Applications*, 186(40):41–46.
- Pressman, R. S. and Maxim, B. R. (2016). *Engenharia de Software: Uma Abordagem Profissional*. AMGH Editora, Porto Alegre, 8 edition.
- Reich, P. and Maalej, W. (2023). Testability refactoring in pull requests: Patterns and trends. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pages 1508–1519. IEEE.
- Rizzi, G., Palomba, F., and Lucia, A. D. (2020). An empirical study on test debt in continuous integration pipelines. In *2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 45–55. IEEE.
- Sommerville, I. (2011). *Engenharia de Software*. Pearson Prentice Hall, São Paulo, 9 edition.
- Valenzuela-Toledo, P. et al. (2024). The hidden costs of automation: An empirical study on github actions workflow maintenance. In *2024 IEEE International Conference on Source Code Analysis and Manipulation (SCAM)*, pages 213–223. IEEE.
- Wang, Y. (2018). Test automation maturity assessment. In *2018 IEEE 11th International Conference on Software Testing, Verification and Validation (ICST)*. IEEE.
- Wang, Y. et al. (2022). Test automation maturity improves product quality—quantitative study of open source projects using continuous integration. *Journal of Systems and Software*, 188:111259.
- Wiklund, K., Eldh, S., Sundmark, D., and Lundqvist, K. (2012). Technical debt in test automation. In *Proceedings of the Third International Workshop on Managing Technical Debt (MTD)*, pages 39–46. IEEE.

Apêndice

Formulação dos Índices de Maturidade

Esta subseção apresenta a formulação completa dos índices sintéticos utilizados neste trabalho. As métricas foram transformadas, normalizadas e combinadas para compor o

Pipeline Maturity Index (PMI), o *Testing Maturity Index (TMI)* e o *Integrated Maturity Index (IMI)*.

$$x_{\log} = \log(1 + x), \quad x \geq 0 \quad (1)$$

$$\text{scale}(m) = 10 \cdot \frac{m - \min(m)}{\max(m) - \min(m)} \quad (2)$$

Quando $\max(m) = \min(m)$, o valor escalado é definido como 5,0 para todos os repositórios.

$$\text{flakyInv} = 1 - \text{clip}(\text{flakyRate}, 0, 1) \quad (3)$$

$$\begin{aligned} PMI = & 0,28 \cdot s(PCL) + 0,12 \cdot s(WCL) \\ & + 0,12 \cdot s(JPL) + 0,10 \cdot s(HMX) \\ & + 0,10 \cdot s(HLT) + 0,12 \cdot s(HCV) \\ & + 0,12 \cdot s(HST) + 0,14 \cdot s(WFL) \end{aligned} \quad (4)$$

$$\begin{aligned} TMI = & 0,18 \cdot s(FCL) + 0,10 \cdot s(UTF) \\ & + 0,10 \cdot s(ITF) + 0,10 \cdot s(ETF) \\ & + 0,14 \cdot s(FWL) + 0,14 \cdot s(TRT) \\ & + 0,12 \cdot s(TPR) + 0,12 \cdot s(FKI) \end{aligned} \quad (5)$$

$$IMI = \frac{PMI + TMI}{2} \quad (6)$$

Símbolo	Descrição
<i>PCL</i>	<i>pipelineComplexity</i> _{log} : complexidade estrutural dos <i>pipelines</i> .
<i>WCL</i>	<i>workflowCount</i> _{log} : quantidade de <i>workflows</i> configurados.
<i>JPL</i>	<i>jobsPerWorkflow</i> _{log} : média de <i>jobs</i> por <i>workflow</i> .
<i>HMX</i>	<i>hasMatrix</i> : presença de <i>build matrix</i> .
<i>HLT</i>	<i>hasLint</i> : presença de verificação de estilo (<i>linting</i>).
<i>HCV</i>	<i>hasCoverageTool</i> : presença de ferramenta de cobertura de testes.
<i>HST</i>	<i>hasSAST</i> : presença de análise estática de segurança.
<i>WFL</i>	<i>wfChurnLast90d</i> _{log} : frequência de modificações nos <i>workflows</i> nos últimos 90 dias.
<i>FCL</i>	<i>testFileCount</i> _{log} : volume total de arquivos de teste.
<i>UTF</i>	<i>unitTestFiles</i> _{log} : arquivos de testes unitários.
<i>ITF</i>	<i>integrationTestFiles</i> _{log} : arquivos de testes de integração.
<i>ETF</i>	<i>e2eTestFiles</i> _{log} : arquivos de testes de ponta a ponta.
<i>FWL</i>	<i>testFrameworkCount</i> _{log} : diversidade de <i>frameworks</i> de teste utilizados.
<i>TRT</i>	<i>testRatio</i> : proporção de arquivos de teste em relação ao total do repositório.
<i>TPR</i>	<i>testPassRate</i> : taxa de aprovação dos testes nas execuções.
<i>FKI</i>	<i>flakyInv</i> : inversão da taxa de testes intermitentes.
<i>PMI</i>	Índice de maturidade de <i>pipelines</i> .
<i>TMI</i>	Índice de maturidade de testes.
<i>IMI</i>	Índice de maturidade integrada.

Tabela 6. Abreviações e índices utilizados nas equações de maturidade

Elemento	Interpretação
Transformação logarítmica	O termo 1 em $\log(1 + x)$ permite transformar métricas com $x = 0$ sem indefinição matemática, além de reduzir assimetria e efeito de <i>outliers</i> .
Normalização	O fator 10 converte a normalização para a escala de 0 a 10, facilitando interpretação e comparação entre métricas heterogêneas.
Caso degenerado da normalização	Quando $\max(m) = \min(m)$, o valor 5,0 é usado como ponto neutro da escala, evitando divisão por zero sem favorecer artificialmente a métrica.
Inversão da instabilidade	O cálculo de <i>flakyInv</i> transforma a taxa de testes instáveis em uma medida positiva de estabilidade, na qual valores maiores indicam menor instabilidade.
Pesos do PMI	Os pesos do PMI somam 1,00, de modo que o índice seja interpretado como uma média ponderada das métricas normalizadas de maturidade de <i>pipelines</i> .
Pesos do TMI	Os pesos do TMI somam 1,00, mantendo interpretação direta como média ponderada das métricas normalizadas de maturidade de testes.
Cálculo do IMI	O IMI é calculado como a média entre PMI e TMI, atribuindo contribuição igual à maturidade de <i>pipeline</i> e à maturidade de testes.

Tabela 7. Interpretação dos parâmetros e pesos das equações de maturidade